

Universidad Da Vinci De Guatemala
Facultad de Ingeniería
Carrera: Ingeniería en Sistemas



FACULTAD DE INGENIERÍA

**UNIVERSIDAD DA VINCI
DE GUATEMALA**

Examen final : Fichas Técnicas de Estructuras Arbóreas y Aplicación Práctica

Juan Carlos Vagas Hernández
202104264
Curso: Estructura de Datos 1

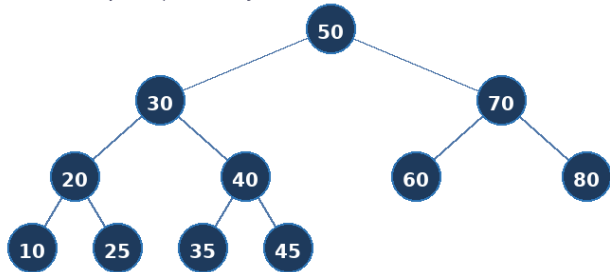
Guatemala, 16 de mayo de 2026

Árbol Binario de Búsqueda (BST)

Ficha Técnica — Estructuras Arbóreas

Visualización Conceptual

BST: valores menores → izquierda | valores mayores → derecha



Análisis de Complejidad (Big O)

Operación	Búsqueda	Inserción	Eliminación
Caso Promedio	$O(\log n)$	$O(\log n)$	$O(\log n)$
Peor Caso	$O(n)$	$O(n)$	$O(n)$

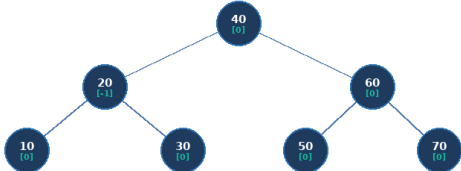
Concepto:	Estructura de datos jerárquica, no lineal y dinámica, compuesta por nodos donde cada uno contiene una clave (valor) y un máximo de dos referencias a otros nodos (hijo izquierdo e hijo derecho). Su propiedad fundamental establece que para cualquier nodo dado, todas las claves del subárbol izquierdo deben ser estrictamente menores a la clave del nodo raíz local, y todas las claves del subárbol derecho deben ser estrictamente mayores
Características	Se basa puramente en punteros/referencias. Al realizar una inserción, se inicia desde la raíz comparando el valor entrante con el del nodo actual de forma iterativa o recursiva. El árbol carece por completo de mecanismos automáticos de reestructuración o balanceo. El peor caso (complejidad lineal) ocurre cuando los datos de entrada se introducen de forma estrictamente ordenada (ascendente o descendente), lo que provoca que el árbol degenera mecánicamente en una lista enlazada simple ("árbol sesgado" o skewed tree).
Ventajas:	Implementación simple. Búsqueda, inserción y eliminación en $O(\log n)$ promedio cuando está balanceado. No requiere sobrecosto de memoria adicional en los nodos más allá de las dos referencias estándar. Muestra una eficiencia óptima cuando las inserciones ocurren en un orden aleatorio
Limitaciones:	Altamente inestable y vulnerable al orden de llegada de los datos. Sin un control externo, pierde por completo su ventaja de eficiencia logarítmica y degenera a $O(n)$, lo que destruye el rendimiento del sistema en conjuntos de datos predecibles u ordenados
Aplicación real: Caso de uso de industria	Se utiliza principalmente en sistemas integrados o aplicaciones en memoria de pequeña escala donde se tiene absoluta certeza de que el flujo de datos de entrada es aleatorio o desordenado, o como el bloque estructural básico sobre el cual se enseñan y construyen arquitecturas arbóreas comerciales auto-balanceada. Un ejemplo puede ser: Bosch (Fabricante de componentes automotrices e IoT). Ya que ellos pueden usar esta estructura para el subsistema de gestión de tablas de calibración de sensores en tiempo real dentro de microcontroladores embebidos. ¿Por qué se usa exactamente? Bueno, en este tipo de hardware industrial la memoria RAM es diminuta (a veces solo unos pocos kilobytes) y no hay espacio para el sobrecosto de punteros adicionales o variables de altura que exige un árbol auto-balanceado. Si el sistema garantiza por diseño físico que los datos de los sensores ingresan con variaciones caóticas/aleatorias, un BST nativo ofrece búsquedas logarítmicas muy veloces con el mínimo consumo de memoria y la menor complejidad de procesamiento posible.

Árbol AVL (Adelson-Velsky & Landis)

Ficha Técnica — Estructuras Arbóreas

Visualización Conceptual

AVL: Factor de equilibrio en cada nodo {-1, 0, 1} | Rotaciones restauran balance automáticamente



Rotaciones: Simple-Izq | Simple-Der | Doble-IzqDer | Doble-DerIzq

Análisis de Complejidad (Big O)

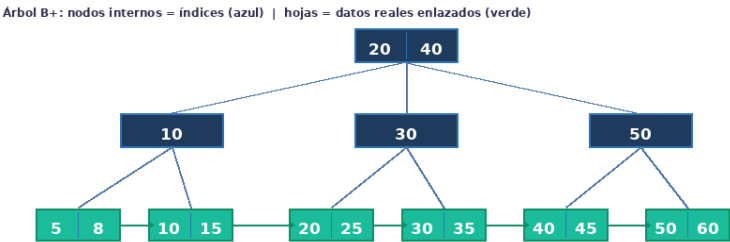
Operación	Búsqueda	Inserción	Eliminación
Caso Promedio:	$O(\log n)$	$O(\log n)$	$O(\log n)$
Peor Caso:	$O(\log n)$	$O(\log n)$	$O(\log n)$

Concepto:	Es un Árbol Binario de Búsqueda auto-balanceado de manera estricta. Nombrado en honor a sus inventores Adelson-Velsky y Landis, introduce una regla rígida de equilibrio estructural: para cualquier nodo del árbol, la diferencia entre las alturas de su subárbol izquierdo y su subárbol derecho no puede ser mayor que uno.
Características	Cada nodo del árbol almacena un entero adicional conocido como "Factor de Equilibrio" (definido como $Altura_Derecha - Altura_Izquierda$), cuyos valores válidos están estrictamente limitados a $\{-1, 0, 1\}$. Tras cada inserción o eliminación, el algoritmo realiza un recorrido hacia atrás (backtracking) recalculando las alturas. Si el factor de equilibrio de algún nodo muta a -2 o 2 , se desencadena de inmediato una reestructuración local mediante operaciones de punteros denominadas "Rotaciones". Existen cuatro tipos de rotaciones: Simples (Izquierda-Izquierda, Derecha-Derecha) y Dobles (Izquierda-Derecha, Derecha-Izquierda).
Ventajas:	Al mantener un balanceo tan estricto, garantiza de forma matemática que el tiempo de búsqueda permanezca en $O(\log n)$ en el peor escenario posible. Ofrece las velocidades de lectura y búsqueda más rápidas entre los árboles binarios dinámicos.
Limitaciones:	Las inserciones y eliminaciones conllevan un sobrecosto de CPU debido al constante cálculo de alturas y ejecución de rotaciones. Requiere almacenamiento extra en cada nodo para mantener el estado de la altura o el factor de balanceo.
Aplicación real: Caso de uso de industria	Bases de datos en memoria orientadas predominantemente a la lectura intensiva o diccionarios del sistema donde la información es estática o cambia con poca frecuencia, pero requiere búsquedas inmediatas y tiempos de respuesta totalmente predecibles. Un ejemplo puede ser: Cloudflare (Proveedores de CDN y Seguridad Web). Ya que ellos pueden usar esta estructura para subsistemas de resolución y filtrado de reglas de Geolocalización IP y Listas Negras (Firewall de red). ¿Por qué se usa exactamente? Bueno, las bases de datos de direcciones IP cambian muy poco (pocas inserciones o eliminaciones al día), pero se consultan miles de millones de veces por segundo cada vez que un usuario intenta acceder a un sitio web protegido. Al ser el AVL el árbol binario con el balanceo más estricto de todos, garantiza un tiempo de búsqueda óptimo en el peor de los casos, asegurando que el tráfico legítimo no sufra retrasos (latencia).

Árbol B / B+

Ficha Técnica — Estructuras Arbóreas

Visualización Conceptual



Análisis de Complejidad (Big O)

Las hojas están conectadas en lista enlazada → consultas por rango eficientes

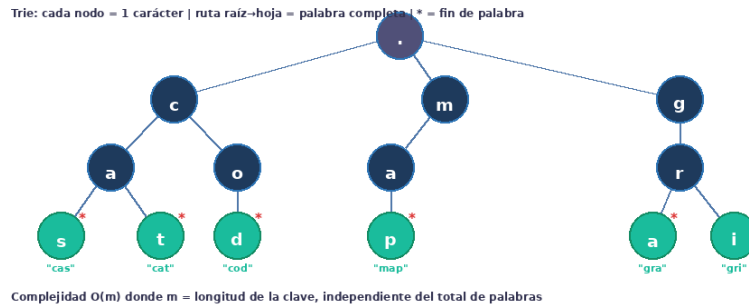
Operación	Búsqueda	Inserción	Eliminación
Caso Promedio:	$O(\log n)$	$O(\log n)$	$O(\log n)$
Peor Caso:	$O(\log n)$	$O(\log n)$	$O(\log n)$

Concepto:	Son estructuras de búsqueda balanceadas de tipo "multi-camino" o multidireccional, diseñadas explícitamente para optimizar sistemas de almacenamiento secundario (discos duros y unidades SSD). A diferencia de las estructuras binarias, un árbol B de orden 'm' permite que cada nodo almacene múltiples claves simultáneamente y posea hasta 'm' nodos hijos, expandiéndose de forma horizontal.
Características:	En el Árbol B tradicional, las claves y los datos reales pueden coexistir en cualquier nivel. En la variante sofisticada denominada Árbol B+, se introduce una separación estricta: los nodos internos <i>solo</i> contienen claves numéricas destinadas a servir como guías de enrutamiento o índices, mientras que los registros y datos reales se almacenan <i>únicamente</i> en las hojas. El mecanismo de balanceo actúa de abajo hacia arriba: cuando un nodo se llena más allá de su capacidad estipulada, se divide en dos (split) y la clave intermedia asciende al nodo padre. Adicionalmente, en los Árboles B+, todas las hojas están conectadas secuencialmente entre sí mediante una lista enlazada.
Ventajas:	Gracias a su alto factor de ramificación, la altura del árbol es extremadamente baja, minimizando drásticamente las costosas operaciones de lectura y escritura en disco (I/O). El enlace secuencial de las hojas en el Árbol B+ permite realizar consultas por rango de manera inmediata y fluida sin hacer backtracking.
Limitaciones:	Su codificación algorítmica e implementación es de una complejidad matemática muy alta. Existe la posibilidad de fragmentación interna o desperdicio de espacio en almacenamiento, dado que las reglas mecánicas permiten que los nodos operen estando llenos solo al 50%.
Aplicación real: Caso de uso de industria	Estructura pilar e indispensable en los sistemas de archivos de los sistemas operativos más importantes (NTFS, ext4, APFS) y en los motores de almacenamiento internos de bases de datos relacionales empresariales. Un ejemplo puede ser: Oracle Corporation (Desarrolladores del motor de bases de datos MySQL/InnoDB). Ya que ellos pueden usar esta estructura para el subsistema de indexación de Claves Primarias (Clustered Indexes) que residen en unidades de almacenamiento secundario (discos duros o SSD). ¿Por qué se usa exactamente? Bueno, cuando buscas registros masivos, la información no está en la memoria RAM, sino en un disco cuyas operaciones de lectura (I/O) son el mayor cuello de botella. El Árbol B+ es ancho y de muy poca altura, almacenando los datos reales únicamente en las hojas inferiores, las cuales están unidas por una lista enlazada. Esto permite a la base de datos hacer una búsqueda vertical muy corta y luego leer secuencias inmensas de datos de forma horizontal, minimizando los costosos accesos físicos al disco.

Trie (Árbol de Prefijos)

Ficha Técnica — Estructuras Arbóreas

Visualización Conceptual



Análisis de Complejidad (Big O)

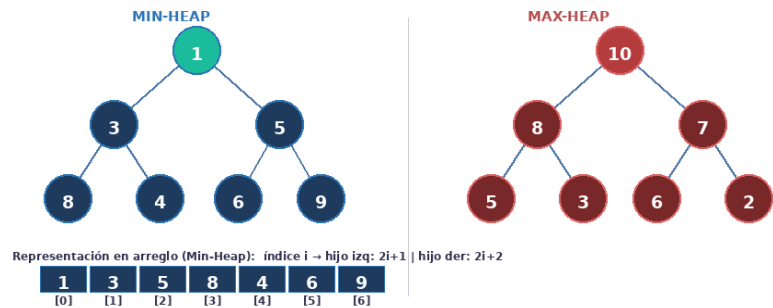
Operación	Búsqueda	Inserción	Eliminación
Caso Promedio:	$O(m)$	$O(m)$	$O(m)$
Peor Caso:	$O(m)$	$O(m)$	$O(m)$

Concepto:	Árbol de búsqueda ordenado especializado en el almacenamiento y recuperación ágil de conjuntos de claves compuestas por secuencias, típicamente cadenas de caracteres (strings). A diferencia de otras estructuras, los nodos de un Trie no guardan la clave completa en su interior; en su lugar, es la posición exacta del nodo dentro de la jerarquía la que define de forma implícita la clave o prefijo asociado
Características:	El nodo raíz actúa siempre como una cadena vacía. Cada nodo subsiguiente representa un único carácter perteneciente a un alfabeto predefinido. Al desplazarse verticalmente por una ruta desde la raíz, se va reconstruyendo la palabra de forma progresiva. Aquellos nodos que marcan la culminación de una palabra semánticamente válida contienen un indicador binario (booleano) especial, usualmente denominado `isEndOfWord`. Mecánicamente, todas las palabras que comparten un mismo prefijo inicial reutilizan los mismos nodos antecesores comunes, optimizando la estructura por superposición.
Ventajas:	Tiempos de búsqueda inigualables y con predictibilidad determinista $O(m)$. No sufre problemas de degradación por colisiones como las tablas hash. Permite listados automáticos y eficientes de elementos basados en prefijos compartidos
Limitaciones:	Consumo de memoria sumamente ineficiente en su diseño nativo básico. Dado que cada nodo individual debe almacenar un arreglo completo de referencias para cada carácter del alfabeto (ej. 26 posiciones para texto estándar en inglés), se generan numerosos punteros vacíos o nulos en memoria (sparse arrays).
Aplicación real: Caso de uso de industria	Autocompletado de búsquedas (Google Search, buscadores de IDEs), corrección ortográfica, routers de red (IP prefix matching). Un ejemplo puede ser: Google (Motor de Búsqueda Web y Tecnología). Ya que ellos pueden usar esta estructura para el subsistema de sugerencias instantáneas en su barra principal de búsqueda (Google Suggest) o en diccionarios de teclados móviles. ¿Por qué se usa exactamente? Bueno, si Google tuviera que comparar la palabra incompleta "estruc..." contra una base de datos tradicional de miles de millones de registros, el sistema colapsaría. En un Trie, la palabra no se guarda como texto, sino que se forma al recorrer la ruta del árbol. Al teclear "e-s-t-r-u-c", el algoritmo simplemente desciende 6 niveles y recupera al instante todas las terminaciones que comparten ese camino común. Su tiempo de respuesta depende únicamente de la longitud de lo que escribes, siendo invulnerable a la inmensa cantidad de palabras totales almacenadas.

Heap — Montículo Min/Max

Ficha Técnica — Estructuras Arbóreas

Visualización Conceptual



Análisis de Complejidad (Big O)

Operación	Búsqueda	Inserción	Eliminación
Caso Promedio:	$O(n)^*$	$O(\log n)$	$O(\log n)$
Peor Caso:	$O(n)$	$O(\log n)$	$O(\log n)$

Concepto:	Es un árbol binario especial con severas restricciones estructurales y de orden, diseñado principalmente para gobernar colas de prioridad eficientes. Se modela conceptualmente como un árbol binario completo, lo que significa que todos los niveles están totalmente llenos de nodos, a excepción posiblemente del último nivel, el cual se va poblando de manera obligatoria de izquierda a derecha
Características:	Se rige bajo la denominada "Propiedad de Heap". En un Max-Heap, el valor de todo nodo padre es mayor o igual que el de sus hijos, ubicando al elemento máximo en la raíz. En un Min-Heap, el padre es menor o igual, ubicando al mínimo en la raíz. Al insertar un elemento, este se coloca mecánicamente al final del árbol (para preservar la completitud) y se ejecuta un proceso de ascenso ("bubble-up" o flotación) mediante intercambios con su padre hasta restituir el orden. Al extraer la raíz, esta se sustituye temporalmente con el último elemento del árbol y se ejecuta un proceso de descenso ("bubble-down" o hundimiento). Debido a que es un árbol binario completo, se implementa físicamente de manera idónea sobre un arreglo secuencial contiguo libre de punteros (donde para un índice i , su hijo izquierdo se halla en $2i + 1$ e hijo derecho en $2i + 2$)
Ventajas:	Acceso instantáneo en $O(1)$ al elemento crítico con mayor prioridad del sistema. Eficiencia espacial inigualable (0% de sobre costo por punteros u objetos dinámicos al mapearse en arreglos).
Limitaciones:	Realizar búsquedas de elementos intermedios o aleatorios es sumamente ineficiente, tomando un tiempo secuencial lineal $O(n)$, ya que no existe un ordenamiento o relación de magnitud definida entre nodos hermanos
Aplicación real: Caso de uso de industria	Algoritmo de Dijkstra (cola de prioridad), Heap Sort, schedulers de sistemas operativos (gestión de procesos por prioridad), colas de eventos en simuladores. Un ejemplo puede ser: Netflix (Plataforma de transmisión de video en streaming). Ya que ellos pueden usar esta estructura para el subsistema de la "cola de prioridad" dentro del búfer de red de sus reproductores de video. ¿Por qué se usa exactamente? Bueno, el video llega a tu pantalla fragmentado en miles de paquetes; algunos traen subtítulos, otros audio, pero los más urgentes son los "cuadros clave" de imagen para evitar que se congele la película. El Heap (ya sea Min o Max) garantiza que el paquete con la más alta urgencia esté absoluta y matemáticamente en la raíz del árbol, permitiendo extraerlo de forma instantánea. Al sistema no le importa en qué orden queden los paquetes menos importantes que están abajo, su único objetivo es no perder ni un milisegundo al obtener el dato más crítico